Adam Johnson
PHYS 3023
10/16/2026

Mechanics Project – Adam Johnson

Problem #1:

A) Part A asked us to find the maximum range of a projectile launched at 45 degrees in a vacuum. The range equation in a vacuum is given as $\frac{v_0^2\, sin(2\theta)}{g} - R = 0$, but since we can assume g and v_0 are 1 we can simplify the equation to be $R = \sin(2\theta)$. The reason that 45 degrees is used as the angle is because $\sin(2\theta)$ is at its max value of one when θ = 45. I used Brent's method to solve these problems, which combines root bracketing, interval bisection, and inverse quadratic interpolation. Brent's method finds where a function equals zero, so I rearranged the range equation into that form before using it in my code. After running the code in Fig 1, the calculated range at θ = 45 degrees is 1 dimensionless unit. Since we assumed g and v_0 were both 1 in our normalized model, the range is expressed in dimensionless units...

```
[34]: from scipy import optimize

#Brents method is what is used here, Brent's method combines root bracketing,
#interval bisection, and inverse quadratic interpolation.

# Import packages
import numpy as np
from scipy.optimize import brentq

# Starting angle
theta = 45

# Convert to radians
theta = np.radians(theta)


# Our equation
def f(R):
    return (R - np.sin(2*(theta)))

# Interval setting for the root finding
R = optimize.brentq(f, 0, 1)

print ( "Range" , R)

Range 1.0
```

Fig 1

B) Part B asked us to use the following equation for the range $\frac{v_{yt}+v_{ter}}{v_{x_t}} R = v_{ter}\tau\left(1 - \frac{R}{v_{x_t}\tau}\right)$ and find the range of θ = 0.75. After rearranging the equation and setting v_0 and g equal to 1 we are left with $\frac{1+sin\,\theta}{cos\,\theta} R + ln\left(1 - \frac{R}{cos\,\theta}\right) = 0$. After using the same code as part A (Fig 2) but inserting our new equation, we are given a range of .4996, which is under half the result in a vacuum. This decrease in range is due to air resistance and shows how much air resistance can affect results.

```
[17]: #Same thing but with Linear air resistance insted

from scipy import optimize

import numpy as np
from scipy.optimize import brentq


theta = 0.75


def f(R):
    return ((np.sin(theta) + 1) / np.cos(theta)) * R + \
           np.log(1 - R / np.cos(theta))

R = optimize.brentq(f, 0.1, 0.9 * np.cos(theta))

print("Range:", R)

Range: 0.49959723732660405
```

Fig 2

Adam Johnson
PHYS 3023
10/16/2026

C) Part C and D involve testing different angles (0.8,0.7, etc.) until we have a good approximation to two significant figures of the angle that produces the maximum range. I completed 8 trials to get an approximation of 0.62 radians which produced a range of 0.5140. Once I had an angle approximation, I graphed what the trajectory would look like and compared it to a projectile in a vacuum launched at the same angle (Fig 3).

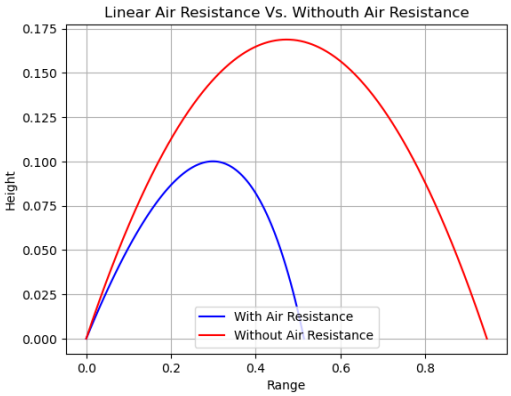


Fig 3

Problem 1 Full Notebook

Problem #2

A) In Problem 2 we are tasked with finding the equations of motion for a cannonball with high initial velocity. This problem differed from Problem 1 by having us use quadratic air drag instead of linear air drag, and this causes air resistance to increase with the square of velocity instead of being directly proportional. The cannonball in part A also has variable air resistance ($c(y) = \gamma D^2 \exp(-y/\lambda)$) so as the height of the projectile increases the drag co-efficient will decrease. I started this problem by finding all the constants and initial conditions used for the problem, and this started with finding mass. To find mass I used the given density and diameter and plugged it into the following equation $M = \rho\left(\frac{\pi D^3}{6}\right)$. After that I used Newton's second law and the drag force equation to find the differential equations for horizontal and vertical motion. These equations came out to $\frac{dv_x}{dt} = -c(y)vv_x/m$, $\frac{dv_y}{dt} = -g - c(y)vv_y/m$ and to solve these I used solve_ivp() in Python (Fig 4). The horizontal range of the cannonball came out to be 2809.7 meters and the trajectory looked like this (Fig 5)

```python
# Solving ODE
def equations(t, state):
    x, y ,vx, vy = state

    v = np.sqrt(vx**2 + vy **2)

    dxdt = vx
    dydt = vy

    dvxdt = -(c(y)/m) * v * vx
    dvydt = -g - (c(y)/m) * v *vy

    return [dxdt, dydt, dvxdt, dvydt]

inital = [x0, y0, vx0, vy0]
```

Fig 4

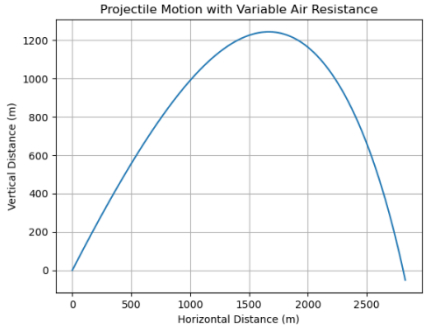


Fig 5

Adam Johnson
PHYS 3023
10/16/2026

B) Part B had us deal with the same cannonball just with two different conditions, one with constant air resistance and another with no air resistance. For the case with constant air resistance the only difference in the code is to set y=0 for the air drag. C(y) now became c(0) = γD^2 and the horizontal range came out to be 2691.80 meters, being approximately a 117.9-meter difference from variable air resistance. The reason for the decrease is that the drag co-efficient is not decreasing with altitude. For the second case with no air resistance the only part of the code that changed was setting c(y) to 0. The horizontal range came out to be 9044.2 meters, which is over 3 times farther than either of the last 2 cases. Below I have compared all 3 trajectories (Fig 6) and the results are obvious as to how much of a difference air resistance makes in objects with high initial velocities

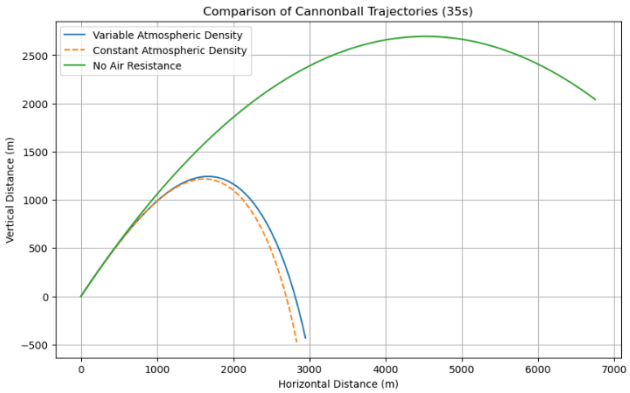


Fig 6

[Problem 2 Full Notebook]

Adam Johnson
PHYS 3023
10/16/2026

Problem #3

Introduction

Problem 3 was the most complicated problem and came with some fun challenges. The task was to model the initial velocity required to send a satellite 80 km into the atmosphere without propulsion during the initial ascent. There have been many startup companies that have been trying to achieve this, and if they're successful it could change much of the aerospace industry. The reason this would be so revolutionary is because not having to use propulsion to launch satellites would cut much of the weight and would be significantly cheaper. When it comes to modeling this launch, this brings some new challenges that make the model more complicated than past problems. For my model, I assumed the satellite would be launched straight up, so the horizontal components of motion and drag could be neglected. I also needed to model air resistance as a function of altitude and velocity rather than assuming it remained constant. Our equation of drag for this problem had us using the Prandtl equation $F(v) = \frac{1}{2}C_w\rho_{air}Av^2$ where C_w is the drag co-efficient and ρ_{air} is air density.

Methods

Firstly, since this is an estimation, I had to make a lot of assumptions to lay the groundwork for the model. Like I stated earlier I assumed that the launch would be directly up, based on the Suborbital Accelerator, a proposed launcher of this kind by the company SpinLanuch. I used the Suborbital Accelerator as a base for a lot of my assumptions on the size of the rocket, because the actual size of the rockets that the Suborbital Accelerator use isn't publicly known. Since I couldn't find any specs on the rockets, I simply used 0.5 meters as the diameter and 400 kg as the mass. Although these values are rough estimates, they provide a reasonable starting point for my calculations. I also assumed that the rocket had a circular cross-sectional area, so I could calculate the area using the following equation $A = \frac{\pi D^2}{4}$. My next assumption would be on the location and date of the flight, as different quantities of the sky can vary from day to day. I randomly picked August 16th, 2026, at 17:10 UTC (12:10 PM Central Time).

Adam Johnson
PHYS 3023
10/16/2026

The next big step was to fully break down the drag equation $F(v) = \frac{1}{2} C_w \rho_{air} A v^2$ so it can be described as an equation of altitude. To do this I needed to model drag co-effect and air density as a function of altitude, which I did by using a preexisting model named NRLMSISE-00. For air density I can directly use NRLMSISE-00 but for drag co-efficient it is a little trickier.

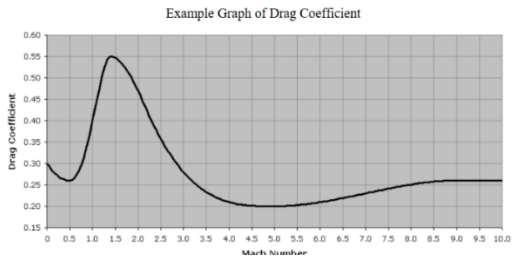


Fig 7

The drag co-efficient depends on Mach number, so I used a chart (Fig 7) provided to me to create a piecewise function that roughly follows the graph. Mach number is given by the following equation $M = \frac{v}{\sqrt{(1.4)(287)T}}$ where T is the temperature, 1.4 is ratio of specific heat, and 287 is the specific gas constant of air. Knowing this, I knew that I needed a model of temperature as a function of altitude which I can use NRLMSISE-00 for.

Now that I have drag co-efficient and air density broken down, I can use NRLMSISE-00 to build out their model. NRLMSISE-00 uses location, date, 81-day average of solar flux, previous days solar flux, and magnetic index. For those last three variables, I used historical solar flux and geomagnetic activity data from CelesTrak's Space Weather Database. After finding that data all I needed to do for drag co-efficient and air density was build their models in Python (Fig 8).

```python
# Atmospheric density and temp
def rho(y):
    densities, temperatures = msise_model(
        date, y / 1000, lat, lon,  f107a, f107, ap
    )
    return densities[5] * 1000

def T(y):
    densities, temperatures = msise_model(
        date, y / 1000, lat, lon, f107a, f107, ap
    )
    return temperatures[1]
```

Fig 8

Now I just needed to model one more thing, that being gravitational acceleration. This one was the simplest of the 3 as I just used the following equation $g = \frac{GM}{(R+y)^2}$ G being the gravitational constant, M being the mass of the earth, R being the radius of the earth, and y being altitude above earth's surface.

The strategy that I used to approximate the velocity needed to send the rocket 80 km was to use Euler's method with a while loop. I did this by writing a while loop (Fig 9) that had short time intervals of 0.01 seconds and ended when the velocity became negative, indicating that the rocket had stopped moving upward. At every time interval, the code would calculate the current air density, temperature, Mach number, drag co-efficient, and acceleration. These values were then used to calculate the new velocity and altitude

```python
    F_drag = 0.5 * Cd(M) * rho(y) * A * v**2

    a = -g - F_drag / m

    y = y + v * dt

    v = v + a * dt

    t = t + dt
```

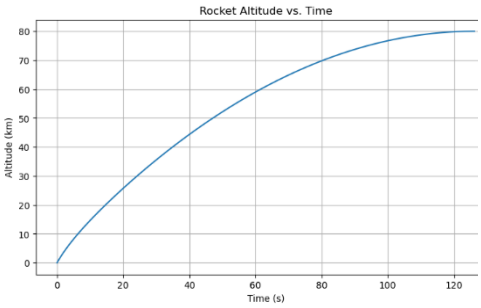
Fig 9

of the rocket. I repeated this process while testing different initial velocities until I found one that got the rocket close to 80 km.

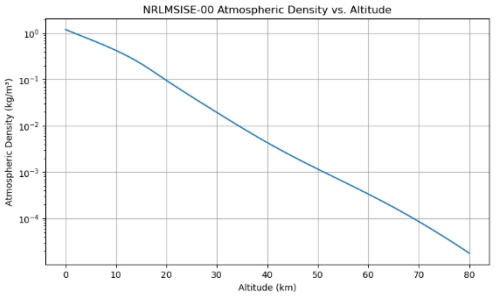
Results

Using this trial-and-error method, it took 9 trials to find a good approximation for the initial velocity required to reach 80 km. My initial trial of 2200 m/s was too high, so I had to gradually decrease the velocity until I found a value that produced the desired altitude. After 9 trials, I approximated that the initial velocity required to send the rocket 80 km into the atmosphere was 2012.7 m/s.

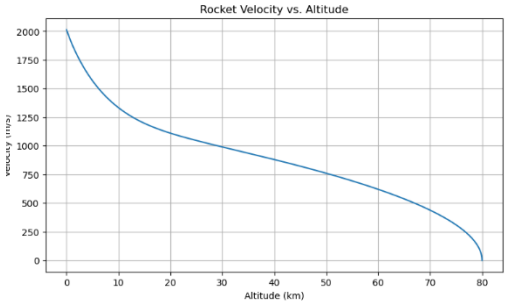
Altitude Vs Time Plot



Air density vs altitude



Velocity vs altituide



Conclusion/discussion

An interesting comparison for these results is to use the same model but without air resistance. For the no-air-resistance model, we can simply use the kinematic equation $V_f^2 = V_0^2 + 2a\Delta y$, and since the launch is completely vertical, we can assume a = -9.81 m/s², V_f^2 = 0, and Δy = 80000. Solving for v_0 we get an initial velocity of 1252.84 m/s. This means that without air resistance, the initial velocity can be 759.86 m/s (60.7% difference) slower and still reach 80 km. This result shows just how much air resistance can affect projectile motion and how air resistance can make our calculations more realistic. One other difference between the two models is that my original model accounted for gravity decreasing with altitude, while the no-air-resistance model assumed gravity remained constant at 9.81 m/s² throughout the flight.

When comparing this problem to Problem 2, it is interesting to see the differences in how air resistance was modeled in the two scenarios. In Problem 2, our model for air resistance used a

simple exponential term that decreased with altitude. In Problem 3, however, since the altitude and velocities were much higher, I used a more in-depth model that accounted for changes in atmospheric density, temperature, and gravity. I also modeled the drag co-efficient as a function of Mach number, meaning it changed as the rocket's velocity and the speed of sound changed. Although the model in Problem 2 was much simpler, the additional factors in Problem 3 allowed me to account for more of the physical conditions affecting the rocket during its ascent.

Some limitations in my results from Problem 3 mainly have to do with the approximations I made about the rocket's size and mass. The assumed diameter of 0.5 meters and mass of 400 kg were rough estimates, and since the actual dimensions and mass of the rockets used by SpinLaunch's Suborbital Accelerator are not readily available, there isn't a great way to make a highly accurate estimate without more information. I also assumed a perfectly vertical launch angle of 90 degrees, which is a simplification of a real launch and would likely need to be adjusted. Another limitation was the piecewise function I created for the drag co-efficient based on Mach number. This was a rough generalization, and modeling the actual drag co-efficient more accurately could improve my results. One other factor that could affect my results is using an even smaller time interval in my numerical calculations. Although I do not expect this to change the results significantly, testing smaller intervals would help confirm the accuracy of my approximation.

One of the main concerns with launches like this is the ethical issue of sending living things through such extreme acceleration. Humans and most animals would not be able to safely withstand these forces, but some biological samples could potentially survive depending on the type of sample and the protective equipment used. The upside is that unmanned launches could be significantly cheaper due to savings on fuel and weight, potentially making launches more frequent and advancing research and other areas of the aerospace industry.

Overall, this was a really fun project that challenged my critical thinking and taught me a lot of useful numerical methods that I can apply to future physics problems.

Problem 3 Full Notebook

Adam Johnson
PHYS 3023
10/16/2026

Bibliography

"BRENTQ#." *Brentq - SciPy v1.18.0 Manual*, 19 June 2026, docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.brentq.html?highlight=scipy%2Boptimize%2Bfsolve&utm

"SOLVE_IVP#." Solve_ivp - SciPy v1.18.0 Manual, 19 June 2026, docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html?highlight=scipy%2Bintegrate%2Brk45%2Bp&utm

"Spinlaunch." *SpinLaunch 2026*, www.spinlaunch.com/. Accessed 20 Oct. 2026.

"Python Interface for the NRLMSISE-00 Neutral Atmosphere Model." *Client Challenge*, pypi.org/project/nrlmsise00/. Accessed 20 Oct. 2026.

"Space Weather Data Documentation." *CelesTrak*, celestrak.org/SpaceData/SpaceWx-format.php. Accessed 20 Oct. 2026.

Hankins, Matthew. Ordinary Differential Equations. Physics 4023, Arkansas Tech University, n.d. Course notes.